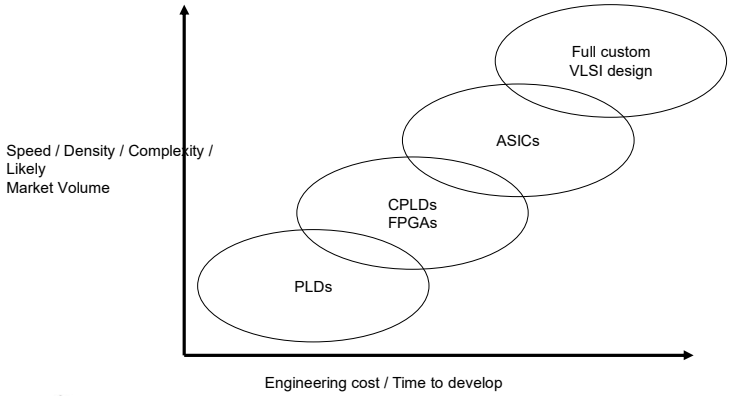
Lojik Devreler ve Tasarım

Bahar 2023

Dr. Öğr. Ü. İhsan Çiçek

6. Donanım Tanımlama Dilleri: Birleşik Lojik Devreler

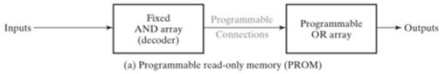
Digital Logic Technology Tradeoffs



PROGRAMMABLE LOGIC TECHNOLOGIES

Simple Programmable Logic Devices

ROM



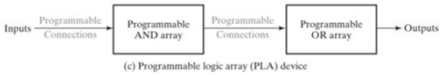
(a) Programmable read-only memory (PROM)

PAL



(b) Programmable array logic (PAL) device

PLA



(c) Programmable logic array (PLA) device

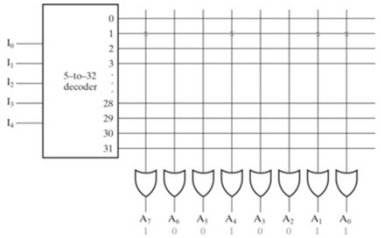
Complex Programmable Logic Devices

CPLD

FPGA

PROGRAMMABLE LOGIC DEVICES

Read-Only Memory (ROM): A read-only memory (ROM) is essentially a device in which “permanent” binary information is stored. The information must be specified by the designer and is then embedded into the ROM to form the required interconnection or electronic device pattern. Once the pattern is established, it stays within the ROM even when power is turned off and on again—that is, ROM is nonvolatile.



PROGRAMMABLE LOGIC DEVICES

Depending on the programming technology and approaches, read- only memories have different names:

1. **ROM**—mask programmed,
2. **PROM**—fuse or anti-fuse programmed,
3. **EPROM**—erasable floating gate programmed,
4. **EEPROM**—electrically erasable floating gate programmed, and
5. **FLASH Memory**—electrically erasable floating gate with multiple erasure and programming modes.

A ROM stores computer programs, in which case the binary code produced by the usual programming tools such as compilers and assemblers is placed in the ROM.

PROGRAMMABLE LOGIC DEVICES

Programmable Logic Array (PLA): does not provide full decoding of the variables and does not generate all the minterms (compact solution).

- An array of AND gates that can be programmed to generate product terms of the input variables is used.
- The product terms are then selectively connected to OR gates to provide the Sop terms for the required Boolean functions. Suitable for SoP/complemented SoP functions.
- Product terms can be shared between the functions, since the same AND gate can be connected to multiple OR gates.

PROGRAMMABLE LOGIC DEVICES

Programmable Logic Array (PLA): For n inputs, k product terms, and m outputs, the internal logic of the PLA consists of

n buffer-inverter gates,

k AND gates,

m OR gates, and

m XOR gates.

There are 2n x k programmable connections between the inputs and the AND array, k x m programmable connections between the AND and OR arrays, and m programmable connections associated with the XOR gates.

PROGRAMMABLE LOGIC DEVICES

Programmable Logic Array (PLA): For n inputs, k product terms, and m outputs, the internal logic of the PLA consists of

n buffer-inverter gates,

k AND gates,

m OR gates, and

m XOR gates.

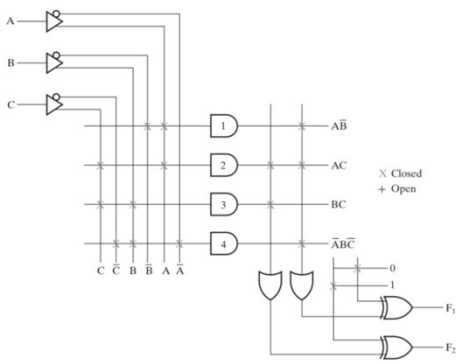
There are 2n x k programmable connections between the inputs and the AND array, k x m programmable connections between the AND and OR arrays, and m programmable connections associated with the XOR gates.

PROGRAMMABLE LOGIC DEVICES

PLA EXAMPLE:

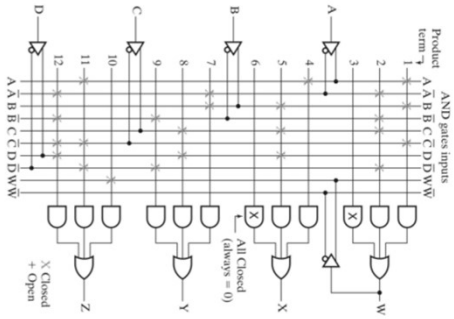
$F_1 = A\overline{B} + AC + \overline{A}B\overline{C}$

$F_2 = \overline{AC} + B\overline{C}$



PROGRAMMABLE LOGIC DEVICES

Programmable Array Logic (PAL): is a PLD with a fixed OR array and a programmable AND array. Because only the AND gates are programmable and cannot be shared by multiple functions, design for the PAL device is easier, but is not as flexible as the PLA.

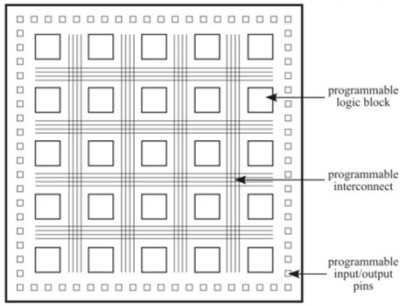


PROGRAMMABLE LOGIC DEVICES

CPLD: consists of multiple PAL like modules combined through a programmable interconnect network.

PROGRAMMABLE LOGIC DEVICES

Field Programmable Gate Array (FPGA): The most common form of programmable logic device currently available is the field programmable gate array (FPGA), most FPGA devices have three programmable elements in common: programmable logic blocks, programmable interconnect, and programmable input/output pins.

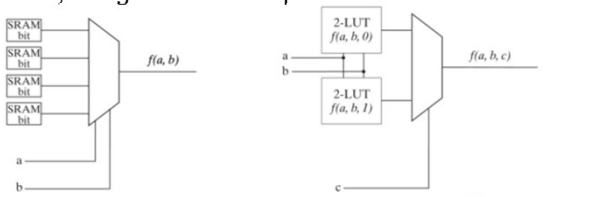


PROGRAMMABLE LOGIC DEVICES

- The advantage of the FPGA is the availability of configurable combinational logic blocks and flip-flops, and ease of re-configuration.
- Most FPGA families are configured using static random access memory (SRAM). FPGAs that use SRAM for their configuration are volatile, meaning that they lose their configuration when power is removed and the configuration must be loaded whenever power is re-applied.
- Other technologies for configuring FPGAs include Flash memory and anti-fuses.

PROGRAMMABLE LOGIC DEVICES

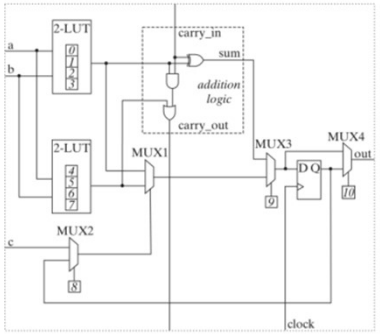
FPGA programmable logic contains combinational and sequential logic that can be configured to implement many different functions. Many FPGA families have programmable logic blocks based upon a look- up table (LUT) to implement combinational functions. A look- up table is a 2^k x 1bit memory that implements the truth table for a function of k variables, referred to as a k-LUT. To implement functions of more than k variables, several k- LUTs can be connected together with multiplexers, using Shannon's expansion theorem



$$f(x_1, x_2, x_3, \ldots, x_k) = x_k \cdot f(x_1, x_2, x_3, \ldots, 1) + \overline{x_k} \cdot f(x_1, x_2, x_3, \ldots, 0)$$

PROGRAMMABLE LOGIC DEVICES

- In addition to LUTs, programmable logic block typically have multiplexers, flip-flops and other logic to provide the ability to configure the block to implement a wide variety of functions.



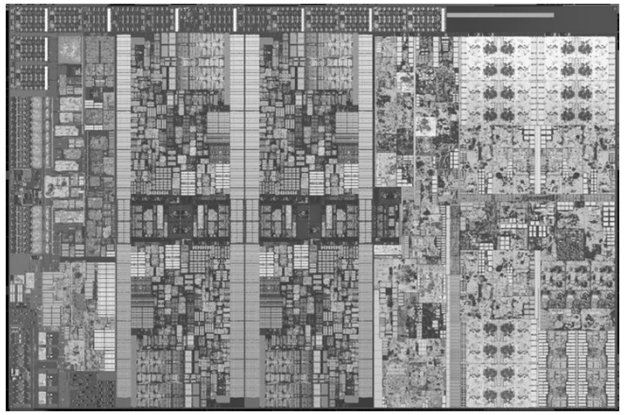
PROGRAMMABLE LOGIC DEVICES

- Programmable interconnection network between the logic blocks provides the wiring between the logic blocks to create circuits that are too large to fit into a single logic block.
- The programmable interconnection network is made up of a set of wires and programmable switches. A programmable switch usually consists of a single NMOS transistor.

PROGRAMMABLE LOGIC DEVICES

- FPGA must be capable of providing a broad range in the number of I/Os depending upon the circuit that is to be implemented.
- FPGA must be compatible with the speed and voltage requirements of the other electrical components to which it will be connected.
- Thus, FPGAs provide a large number of pins that can be configured to be either inputs or outputs, and that can be configured to support a number of different electrical interface standards TTL, CMOS LVTTL, LVCMOS, LVDS etc.

Intel Kaby Lake, 7th Gen, 14nm, 2017



https://en.wikichip.org/wiki/intel/microarchitectures/kaby_lake

How to manage this complexity?

Hardware description languages (HDLs)

- specify complex designs
- keep track of changes and manage within teams
- produce synthesizable code
- (easily) simulate and verify for functional and timing operation

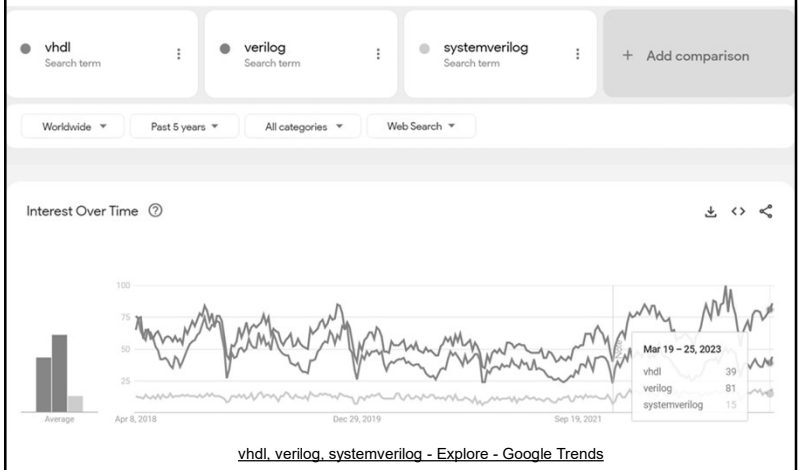
Hardware Description Languages

There are two well-known ones

- VHDL (VHSIC Hardware Description Language)
  - 1981 by US DoD, 1987 IEEE standard - **IEEE 1076-1987**
  - 1993, 2002, 2008, 2019 Updates to standard (**IEEE 1076-2019**)
- Verilog
  - 1984 by Gateway Design Automation
  - 1995 IEEE standard - **IEEE 1364-1995**
  - 2001, 2005 Updates to standard (**1364-2005**)
- SystemVerilog
  - Superset of Verilog
  - 2002 by Co-Design Automation
  - 2005 IEEE standard - IEEE 1800-2005
  - 2009 Merged with Verilog standard - **IEEE 1800-2009**
  - 2017 Update to the standard (**IEEE 1800-2017**)

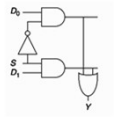
Most commercial designs built using HDLs!!!

trends.google.com - April 2023



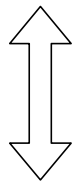
vhdl, verilog, systemverilog - Explore - Google Trends

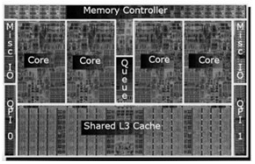
Hierarchical Design



How can we go from a single gate to a complex IC?

- We need to stop the **abstraction** at some level, and this level is usually **logic gates**.
- Define **modules** out of these gates such as *a MUX*.
- Define bigger components out of these modules such as *an ALU*.





How can we approach the design?

- Top-Down Design Methodology
- Bottom-Up Design Methodology

Design Methodologies

Top-Down Design Methodology

- Define **top-level** module and identify its building blocks as **sub-modules**
- Define sub-modules, and their sub-modules, .. until the smallest unit is defined.
- Smallest unit is called a **leaf-cell**. This can be a combination of logic cells, or a technology dependent cell library.

Bottom-Up Design Methodology

- Define the **basic building blocks**. (leaf-cells)
- Using these building blocks, create **bigger modules.** (sub-modules)
- Using these modules, create a **top-level module**.

Module in SystemVerilog

A **module** is the basic building block in **SystemVerilog.**



It requires <u>three</u> properties:

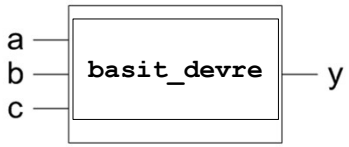
1. Module name (e.g. example)
2. Port names (e.g. a, b, c, y)
3. Port directions (e.g. input / output)

After the module definition, we define the **functionality** of the module.

1. **Behavioral** describe what a module does
2. **Structural** describe how it is built from simpler modules

Module Definition in SystemVerilog

- start with **module**, and end with **endmodule** keywords
- **logic** keyword is necessary to define the type of the signal

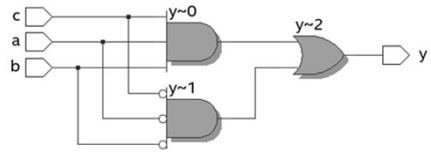



```
/* basit_devre.sv
 * SystemVerilog basit bir devre örneği
 */
module basit_devre (
       input  logic a, b, c,
       output logic y
);
// Devre tanımı buraya gider

endmodule
```

Three properties
1. **Module name**
2. **Port names**
3. **Port directions**

Behavioral design



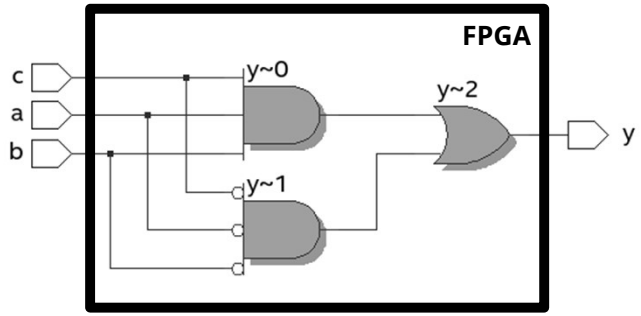

```
/* basit_devre.sv
 * SystemVerilog basit bir devre örneği
 */
module basit_devre (
       input  logic a, b, c,
       output logic y
);
assign y = a & b & c | ~a & ~b & ~c;

endmodule
```

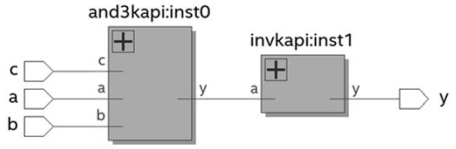
assign keyword is used to define the combinational logic assignments.

FPGA view of the circuit

Here is how the FPGA will look like after synthesize & **fitting** and programming the board.



Structural Design




```
module invkapi
(
input  logic a,
output logic y
);

assign y = ~a;

endmodule
```

```
module and3kapi (
input  logic a, b,
c,
output logic y
);

assign y = a & b &
c;

endmodule
```

```
module nand3kapi (
input  logic a, b, c,
output logic y
);

logic n1;

and3kapi inst0(a, b, c,
n1);
invkapi  inst1(.a(n1),
.y(y));

endmodule
```

Structural Design

Two ways to instantiate.

- **Implicit** (map ports based on **order**)
- **Explicit** (map ports based on **port name**)

Implicit instantiation is easier to make **mistakes**. Go with **explicit**.

```
module and3kapi (
input  logic a, b, c,
output logic y );
```

```
module invkapi(
input  logic a,
output logic y);
```

```
module nand3kapi (
   input  logic a, b, c,
   output logic y
);
logic n1;
and3kapi inst0(.a(a), .b(b), .c(c),
.y(n1));
invkapi  inst1(.a(n1), .y(y));
endmodule
```

```
module nand3kapi (
input  logic a, b, c,
output logic y
);
logic n1;
and3kapi inst0(a, b, c,
n1);
invkapi  inst1(n1, y);
endmodule
```


SystemVerilog Basic Syntax

- SystemVerilog is **Case sensitive**
  - **reset** and **Reset** are not the same signal.
- Names cannot start with numbers
  - **2mux** is an invalid name
- **Whitespace**s are ignored
- C style comments:
  - `// single line comment`
  - `/* multiline comment */`

How about multi-bit ports?

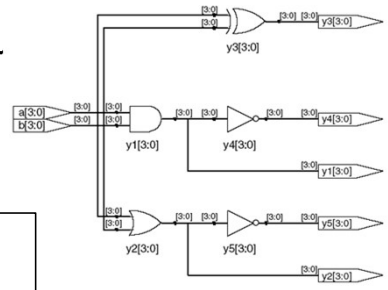
- It becomes difficult to keep track of all the port names, so we **group** the same purpose bits into a single **multi-bit** port, also called **a bus**.
- Before the port names, define the range between **[ ]** with the syntax **`[range_end:range_start]`**
  - Total number of bits: **`range_end - range_start + 1`**

```
module basit_devre2 (
      input  logic [3:0] a, b // a[3], a[2], a[1], a[0],
b[3], ... , b[0]
      output logic [1:0] y    // y[1], y[0]
);

endmodule
```



Bitwise Operators



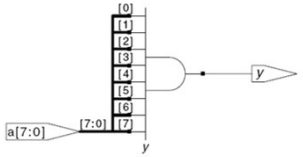

```
module gates (
input  logic [3:0]  a, b,
output logic [3:0] y1, y2, y3,
y4, y5
);
   assign y1 = a & b;    // AND
   assign y2 = a | b;    // OR
   assign y3 = a ^ b;    // XOR
   assign y4 = ~(a & b); // NAND
   assign y5 = ~(a | b); // NOR
endmodule
```

Reduction Operators

```
module and8 (
input  logic [7:0] a,
output logic       y );

   assign y = &a;
   // &a is much easier to write than
   // assign y = a[7] & a[6] & a[5] & a[4] &
   //            a[3] & a[2] & a[1] & a[0];

endmodule
```



It is also applicable to OR and XOR gates.

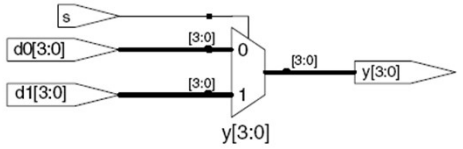
Conditional Assignments

```
module mux2 (
    input  logic [3:0] d0, d1,
    input  logic       s,
    output logic [3:0] y
);

   assign y = s ? d1 : d0;
   // if s is 1, then y = d1, otherwise y = d0

endmodule
```

- **?** is also called a **ternary operator** since it operates on <u>three</u> inputs



SystemVerilog operators

& - Bitwise AND	**<** - Less than
| - Bitwise OR	**<=** - Less or equal
~ - Bitwise negation (NOT)	**>** - Greater than
^ - Bitwise XOR	**>=** - Greater or equal
|| - Logical OR	
&& - Logical AND	**+** - Addition
! - Logical negation	**-** - Subtraction
<< - Left Shift	
>> - Right Shift	***** - Multiplication
== - Equality	**/** - Division
!= - Inequality	**%** - Remainder

FPGAs usually come with fast carry chains to implement these efficiently.

Careful with these. Since we describe hardware, the resulting logic might be huge.

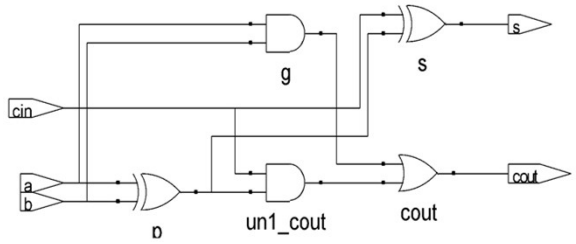
Internal Variables

```
module fulladder (input  logic a, b, cin,
                  output logic s, cout);
  logic p, g;   // internal nodes

  assign p = a ^ b;
  assign g = a & b;

  assign s = p ^ cin;
  assign cout = g | (p & cin);
endmodule
```

Q. What will be the **order of** the assignments? (or which **assign** will happen first?)



Precedence

Highest	~	NOT
	*, /, %	mult, div, mod
	+, -	add,sub
	<<, >>	shift
	<<<, >>>	arithmetic shift
	<, <=, >, >=	comparison
	==, !=	equal, not equal
	&, ~&	AND, NAND
	^, ~^	XOR, XNOR
	|, ~|	OR, NOR
Lowest	?:	ternary operator

Number Representations

N'Bxx

N = number of bits, **B** = base, **xx** = value
N'B is optional but recommended (**default is decimal**)

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101
'b11	unsized	binary	3	00…0011
8'b11	8	binary	3	00000011
8'b1010_1011	8	binary	171	10101011
3'd6	3	decimal	6	110
6'o42	6	octal	34	100010
8'hAB	8	hexadecimal	171	10101011
42	unsized	decimal	42	00…0101010

Bit Manipulations

- By **slicing**, **concatenating** and **duplicating** signals, we can create **a new** multi-bit signals.

```
assign y = {a[2:1], {3{b[0]}}, a[0], 6'b100_010};

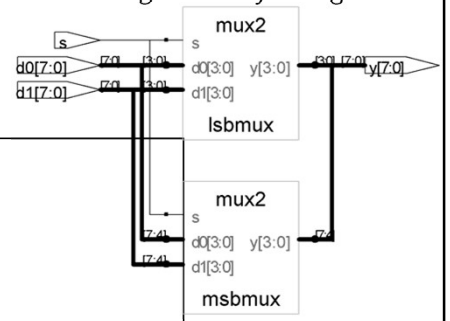
// if y is a 12-bit signal (y[11:0]), the above statement
produces:
//    y = a[2] a[1] b[0] b[0] b[0] a[0] 1 0 0 0 1 0

//    i.e. y[11] = a[2], y[10] = a[1], y[9] = b[0], ...,
y[1] = 1, y[0] = 0

// underscores (_) are used for formatting only to make
// it easier to read. SystemVerilog ignores them.
```


Bit Manipulations II

- We can also divide the signals and assign them by slicing.




```
module mux2_8 (
input  logic [7:0] d0, d1,
input  logic       s,
output logic [7:0] y
);

mux2 lsbmux(d0[3:0], d1[3:0], s, y[3:0]);
mux2 msbmux(d0[7:4], d1[7:4], s, y[7:4]);

endmodule
```

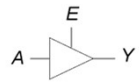
Z: Floating Output

- We can also use **Z** state for a tristate buffer operation.

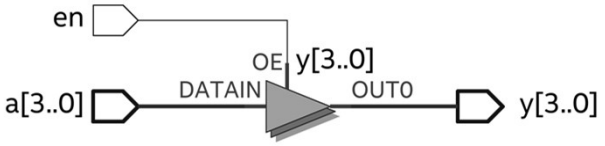
```
module tristate (input  logic [3:0] a,
                 input  logic       en,
                 output logic [3:0] y);

   assign y = en ? a : 4'bz;

endmodule
```



E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1



Parameterized Modules

- What if we want to instantiate multiple, multi-port circuit with different port numbers?

```
module mux2
    #(parameter width = 8)  // name and default value
    (input  logic [width-1:0] d0, d1,
     input  logic             s,
     output logic [width-1:0] y);

  assign y = s ? d1 : d0;
endmodule
```

```
module myawesomecircuit (
...
// Instance with 8-bit bus width (uses default):
  mux2 inst0(d0, d1, s, out);

// Instance with 12-bit bus width:
  mux2 #(12) inst1(d0, d1, s, out);
...
endmodule
```


Delay

- We can specify delay on SystemVerilog using the **#number** syntax and giving **a time unit**

```
`timescale 1ns/1ps

module basit_devre (input logic a, output logic b);
logic n1;
assign #4 n1 = a & b; // assign a and b after 4ns to n1
assign #3 b = ~n1; // assign inverted n1 after 3ns to b

endmodule
```

- Delay is **only** for **simulation**
- **Not synthesizable**
- Used for delay modeling in circuits

HDL to Gates

Synthesis

- Transforms HDL code into a **netlist** describing the hardware (i.e., a list of gates and the wires connecting them)
- Tools **map** these netlists into FPGA hardware.
- Not always possible or optimal.

Simulation

- Tests the circuit operation **without** manufacturing
- Inputs applied to circuit
- Outputs checked for correctness
- Millions of dollars saved by debugging in **simulation** instead of hardware

HDL Tips

- Define a naming scheme and stick to it.
  - Eg.: **i_a, i_b, o_c** or **in_a, ou_b** or **a_i, b_o**
- While defining a bus use **MSB:LSB** order, more intuitive.
  - **Eg.:** instead of **a[0:31]** use **a[31:0]**.
- Create only one module in one file. Easier to organize.
- Make the name of the module and filename the same.
  - Eg.: **alu_devresi.sv** file has **alu_devresi** module.
- **Don't forget** HDL is used **to define and construct hardware**. We describe **how the hardware is connected** and **how it operates**.
  - Don't confuse it with programming languages.

Simulation

- Devrenin doğru çalışıp çalışmadığını test etmek için, test edilen devrenin girişlerine farklı sinyaller ve örnekler göndermek ve çıkışları gözlemek için kurulan devreye **testbench** denir.
  - Ayrı bir devre - genelde port barındırmaz.
  - Test edilen devreyi çağırıp, **port bağlaması** yapılır. Test edilen devreye **design under test (dut)** veya **unit under test (uut)** adı verilir.
- Bu testbench devresi **sentezlenemez**.
- Test metodolojisine göre basit, veya otomatik kontrole dayalı kullanılabilir.
- **İyi** bir **testbench**, devredeki **hataları** bulmak için **büyük önem** taşır.

Simulation

- Commercial simulators
  - **ModelSim and Questa** (Mentor Graphics)
  - **Incisive Enterprise Simulator** (Cadence)
  - **VCS** (Synopsys)
  - Active-HDL/Riviera-PRO (Aldec)
  - Vivado Simulator (Xilinx)
- Open-source simulators
  - Icarus Verilog
  - Verilator
  - GHDL
- We will use **ModelSim Quartus Edition** from **Intel**

Example testbench circuit

```
// 1ns birim zaman, ve 1ps çözünürlükle çalıştır
`timescale 1ns/1ps

module tb_basit_devre ();
    // basit_devre ye bağlanacak giriş ve cıkışlar
    logic a, b, c;
    logic y;

    // test edeceğimiz devreyi bağlayalım
    basit_devre dut0(a, b, c, y);

    // girişleri, 10 birim zaman aralıklarla değiştir
        // initial blokunun içine test paternlerini ekleyebiliriz
    initial begin
        a = 0; b = 0; c = 0; #10; // a, b ve c yi gönder ve 10 ns bekle
        c = 1;               #10; // sadece c yi değiştir ve 10 ns bekle
        a = 1; c = 0;        #10;
        b = 1; c = 1;        #10;

        $stop;                    // simülasyon bittiğinde sonlandir
    end
endmodule
```

Example testbench - autocontrol

```
// 1ns birim zaman, ve 1ps çözünürlükle çalıştır
`timescale 1ns/1ps

module tb_basit_devre ();
    ...
    // test edeceğimiz devreyi bağlayalım
    basit_devre dut0(a, b, c, y);
        ...
    initial begin
        a = 0; b = 0; c = 0; #10;
        if (y !== 1) $display("000 failed.");
        c = 1;               #10;
        if (y !== 0) $display("001 failed.");
        a = 1; c = 0;        #10;
        if (y !== 0) $display("100 failed.");
        b = 1; c = 1;        #10;
        if (y !== 1) $display("111 failed.");

        $stop;                    // simülasyon bittiğinde sonlandir
    end
endmodule
```


Remarks

- **SystemVerilog** veya **VHDL** geniş bir dil, bütün ayrıntılarına girecek vaktimiz yok. Sadece işimize yarayacak kadarıyla uğraşacağız.
- Bir dili öğrendiğiniz zaman **diğer dile geçiş yapması** çok zor değil.
- Aynı programlama dilleri gibi HDL leri de **pratik** yaparak öğrenebilirsiniz.
- Her yazdığınız kodun bir **donanıma** dönüşeceğini unutmayın. O yüzden mutlaka **kontrol** edin.
- Artık devreleri HDL yardımıyla gösterebildiğimiz ve testbench yazabildiğimiz için lablarda bu yolu kullanacağız.
- ModelSim kullanmakla alakalı bir döküman paylaşıldı, labı yapmak için onu takip edin.

END of LECTURE

- Thanks